EMBEDDED SYSTEM DESIGN EXPERIMENT

嵌入式系统设计实验

曾毓

实验 04 RTOS任务管理实验

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

多任务系统基本概念——任务及任务管理

FreeRTOS中关于任务及任务管理的原版及CMSIS封装函数如下所示：

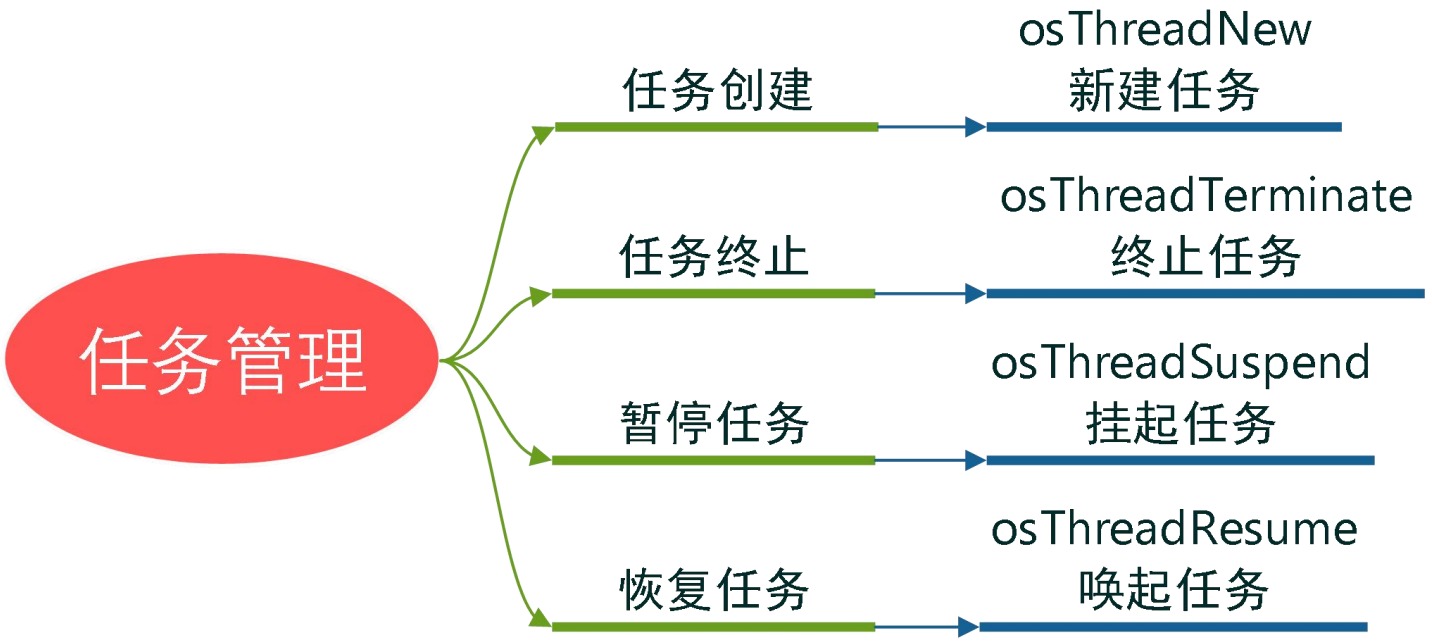
功能分类	FreeRTOS原版函数	CMSIS v2封装函数	说明
任务管理	xTaskCreateStaticxTaskCreate	**osThreadNew**	**创建一个任务并将其添加到活动任务**
	pcTaskGetName	osThreadGetName	获取任务的名称
	xTaskGetCurrentTaskHandle	osThreadGetId	返回当前运行任务的任务ID
	xTaskGetSchedulerState	osThreadGetState	获取任务的当前任务状态
	vTaskPrioritySet	osThreadSetPriority	更改任务的优先级
	uxTaskPriorityGet	osThreadGetPriority	获取任务的优先级
	vTaskSuspend	**osThreadSuspend**	**暂停执行一个任务**
	vTaskResume	**osThreadResume**	**恢复任务的执行**
	vTaskDelete	**osThreadExit**	**终止当前正在运行的任务**
	portYIELD	osThreadYield	将控制权交给下一个就绪态线程
	vTaskDelete	**osThreadTerminate**	**终止任务的执行**
	uxTaskGetStackHighWaterMark	osThreadGetStackSpace	获取任务的可用堆栈空间
	uxTaskGetSystemState	osThreadEnumerate	枚举活动任务

多任务系统基本概念——任务及任务管理

osThreadSuspend

可以让任务A挂起，保持在阻塞状态，直到其他任务使用**osThreadResume**唤起任务A

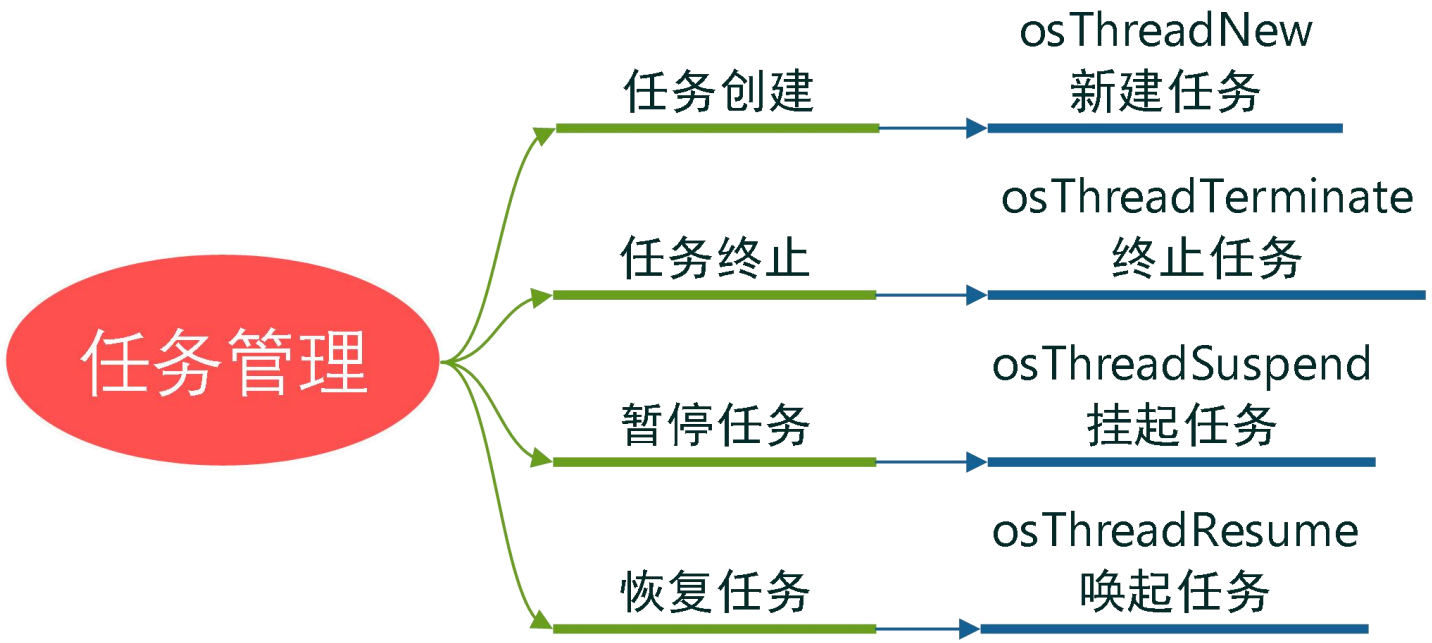
如果其他任务的优先级低于任务A，那么唤起任务A的瞬间系统内核就会上下文切换至任务A，即马上运行。



多任务系统基本概念——任务及任务管理

程序设计时，如果要暂停某个任务运行而后在某个时刻又恢复该任务的运行

- 应该使用osThreadSuspend暂停任务而不是用osThreadTerminate终止任务
- 因为终止任务后只能用osThreadNew再新建一个任务，然而新建任务的内部变量值并不能恢复到终止任务时的状态。



多任务系统基本概念——任务通知

FreeRTOS的每一个任务都有一个32位的通知（标志）值：

- 用户可以用这个通知值保存一个32位（非0）整数或指针值
- 多数情况下，任务通知可以实现任务间简单通信和任务同步功能。
- 使用任务通知不需要另外创建通知变量，更为灵活。

FreeRTOS的CMSIS v2封装中，任务通知又称为线程标志（ThreadFlag），任务通知的常用函数如下表所示：

功能分类	FreeRTOS原版函数	CMSIS v2封装函数	说明
任务通知	xTaskNotify xTaskNotifyFromISR	**osThreadFlagsSet**	**设置指定任务的标志**
	xTaskNotifyAndQuery xTaskNotify	osThreadFlagsClear	清除当前任务的标志
	xTaskNotifyAndQuery	osThreadFlagsGet	获取当前任务的标志
	xTaskNotifyWait	**osThreadFlagsWait**	**等待当前任务的一个或多个任务标志位被设置过**

多任务系统基本概念——任务通知

CubeMX自动生成的代码中为每一个任务都定义了一个任务ID变量（或称为任务句柄）。如下示例，使用任务通知时，在发送通知的任务中调用Set函数发送通知给指定ID的任务，然后在对应ID的任务中使用Wait函数获取并清除任务通知。

```
// 默认任务id变量（CubeMX自动生成）
osThreadId_t defaultTaskHandle;
// 定义4个按键组合掩码
#define KEY_MASK    K1_Pin | K2_Pin | K3_Pin | K4_Pin
```

```
// 在按键扫描任务向默认任务发送通知
void StartKeyTask(void *argument) {
  for(;;) {
    ...
    // 将非0键码值作为任务通知发给默认任务
    osThreadFlagsSet(defaultTaskHandle, key);
    ...
  }
}
```

```
// 在默认任务等待任务通知，
void StartDefaultTask(void *argument) {
  for(;;) {
    ...
    // 等待按键掩码中任一位上的任务通知，最多等10ms
    int flag = osThreadFlagsWait(
                    KEY_MASK, osFlagsWaitAny, 10);
    if (flag > 0) {  // 等待到有效的任务通知
      ...
    }
    ...
    osDelay(1);
  }
}
```

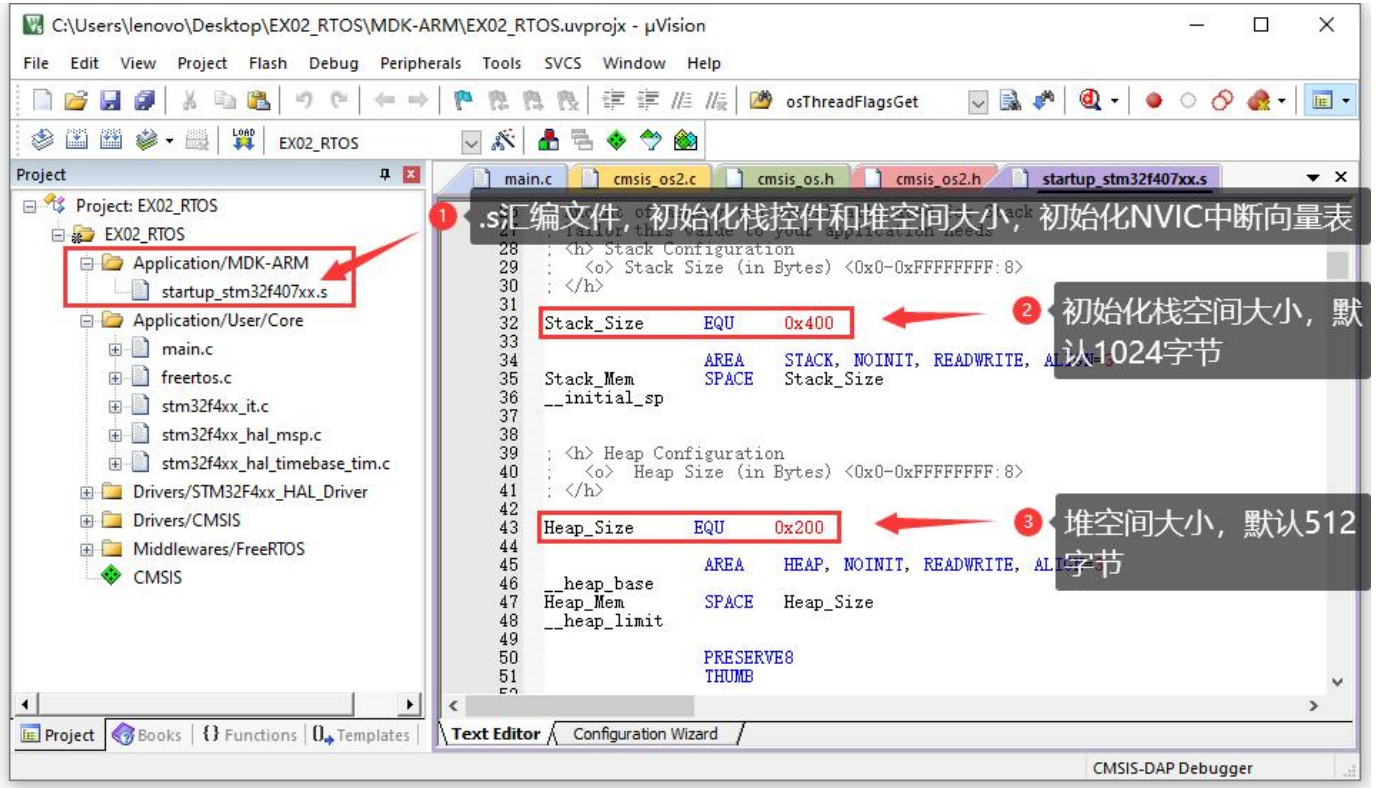
多任务系统基本概念——内存管理

如图所示，MDK工程的汇编初始化文件中指定了系统初始化堆栈大小。

在main函数中定义临时变量和用malloc函数申请动态内存时：

- **需要注意变量占用空间大小不超过初始化汇编文件中指定的堆栈大小。**

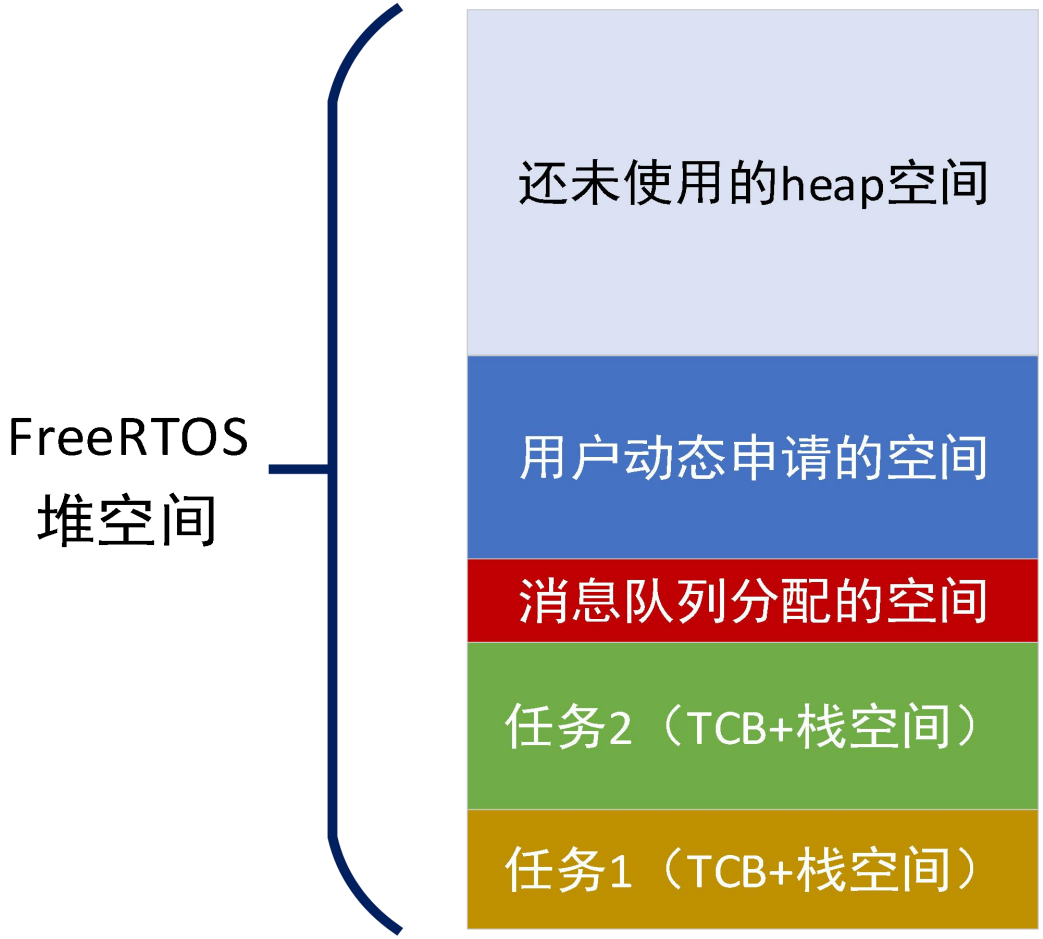
初始化文件默认的堆栈大小都比较小，有需要的情况下需要适当调大，CubeMX中导出MDK工程时也可以设置这两个值，不需要打开汇编文件手动修改。



多任务系统基本概念——内存管理

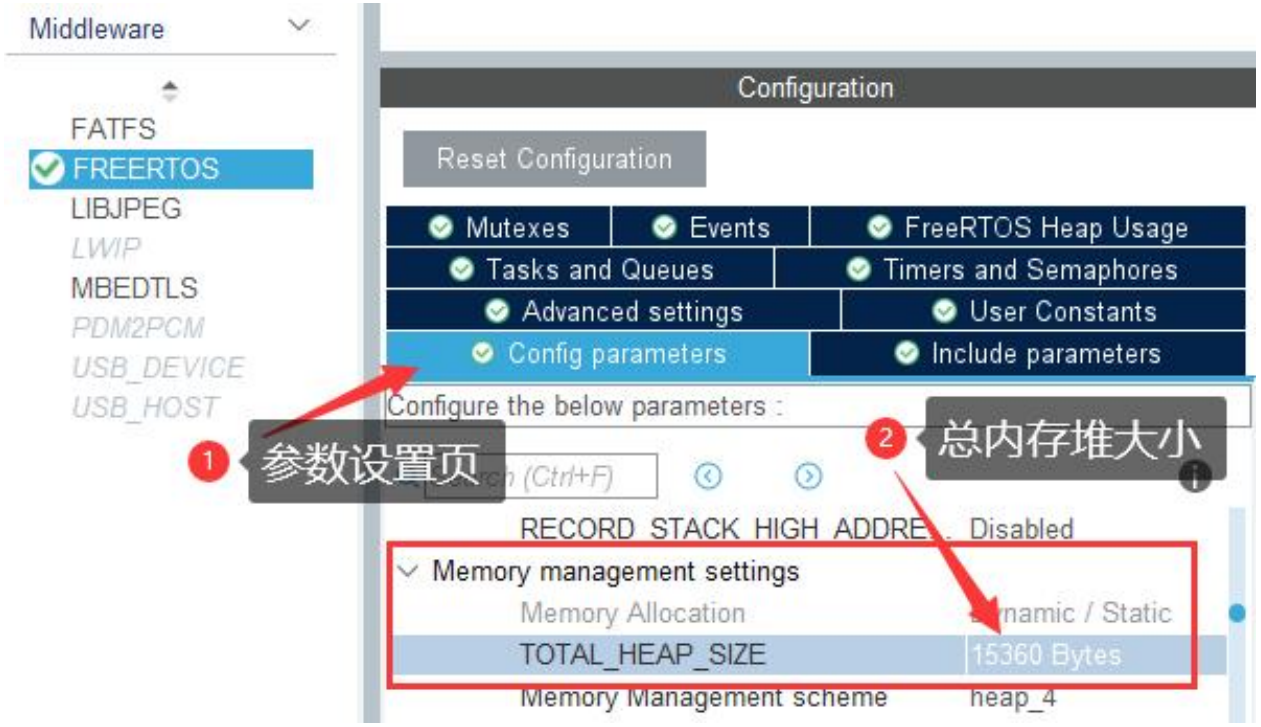
当CubeMX中开启了FreeRTOS操作系统后，用户在任务函数中声明的临时变量和动态申请内存就分配到了FreeRTOS内存堆空间上了。

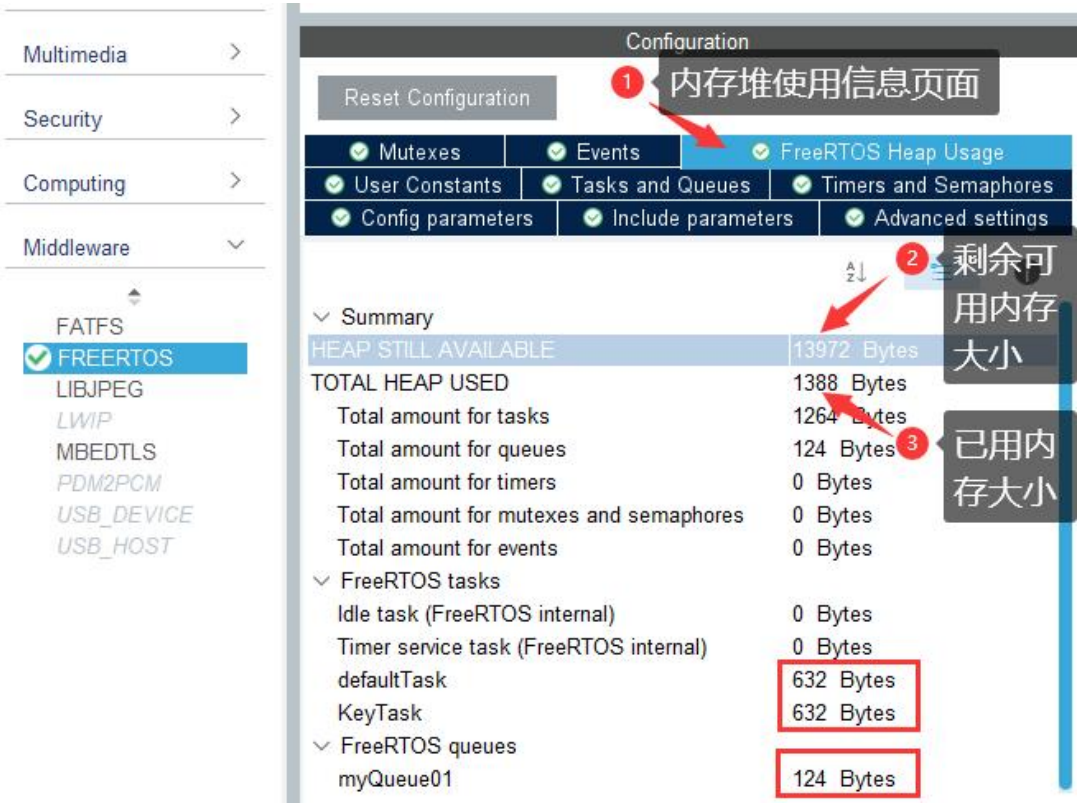
- TCB是任务控制块（Task Control Block）结构体的简称
  - 每个任务都有一个TCB
  - 其大小固定，约占120字节。
- 用户在任务函数中动态申请的内存分配在任务栈空间之外（图中蓝色部分）
- 任务函数中声明的临时变量分配在各个任务的栈空间内。



多任务系统基本概念——内存管理

如图，在CubeMX的FreeRTOS参数设置页面，可以设置FreeRTOS总的堆空间大小（TOTAL_HEAP_SIZE）。在堆空间使用情况页面中可以看到各个任务所用内存大小，以及可以动态申请的内存空间大小。

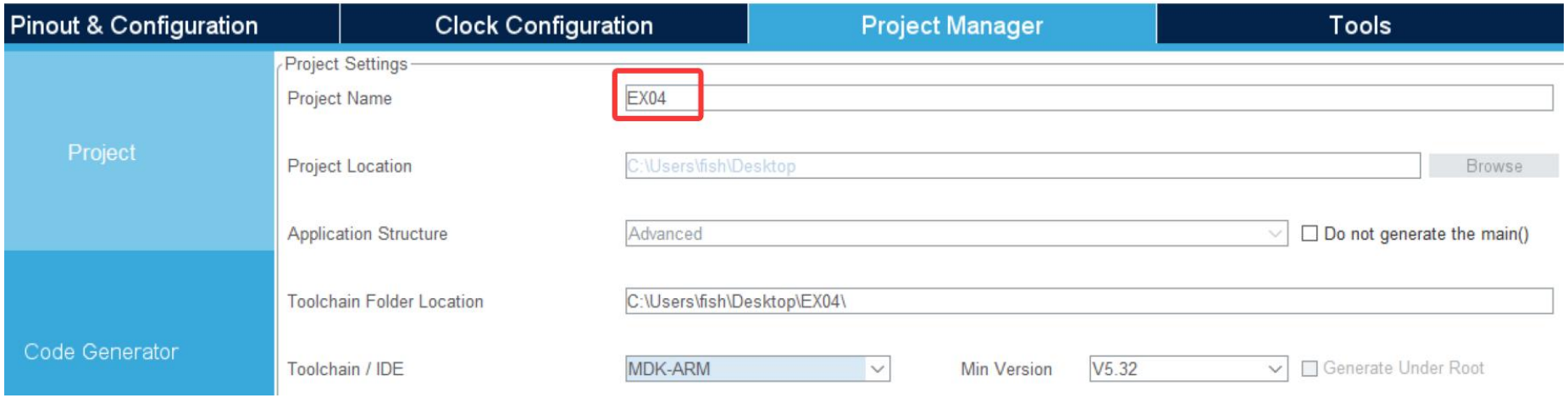




FreeRTOS应用示例——按键控制流水灯

本次实验将演示一个按键控制流水灯示例应用，该示例使用FreeRTOS将按键扫描动作和流水灯控制分配在两个任务中独立运行，按下按键时使用任务间通信方法通知流水灯控制有按键动作。

参考教材4.4节内容，在CubeMX中选择器件STM32F407VE开始一个新的CubeMX工程。然后参照教材图4-15～图4-21配置系统时钟、LED和按键端口并添加FreeRTOS及其任务。最后导出MDK工程如下所示，设置工程名称为EX04，并参照图4-23设置代码生成选项。



FreeRTOS应用示例——编写LED和按键驱动

使用KEIL MDK软件打开EX04工程，打开左侧工程文件列表中的**gpio.c**源文件，添加LED亮灯函数和按键扫描函数：

```
/* USER CODE BEGIN 2 */
void SetLeds(uint8_t dat) {
    // LED亮灭控制，低电平亮，高电平灭
    // 8个灯对应dat的低8位
    HAL_GPIO_WritePin( L1_GPIO_Port, L1_Pin,
        (dat & 0x01) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L2_GPIO_Port, L2_Pin,
        (dat & 0x02) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L3_GPIO_Port, L3_Pin,
        (dat & 0x04) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L4_GPIO_Port, L4_Pin,
        (dat & 0x08) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L5_GPIO_Port, L5_Pin,
        (dat & 0x10) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L6_GPIO_Port, L6_Pin,
        (dat & 0x20) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L7_GPIO_Port, L7_Pin,
        (dat & 0x40) ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin( L8_GPIO_Port, L8_Pin,
        (dat & 0x80) ? GPIO_PIN_RESET : GPIO_PIN_SET);
}
```

```
uint16_t ScanKey(void) {
    // 按键按下时，K1 ~ K4低电平有效（篇幅有限，K5、K6检测可参考教材P69）
    uint8_t key = 0;
    if (HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin) == GPIO_PIN_RESET)    key |= K1_Pin;
    if (HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin) == GPIO_PIN_RESET)    key |= K2_Pin;
    if (HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin) == GPIO_PIN_RESET)    key |= K3_Pin;
    if (HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin) == GPIO_PIN_RESET)    key |= K4_Pin;

    if (key > 0){
        osDelay(10);          // 按键延时10毫秒消抖
        uint8_t key2 = 0;
        if (HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin) == GPIO_PIN_RESET) key2 |= K1_Pin;
        if (HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin) == GPIO_PIN_RESET) key2 |= K2_Pin;
        if (HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin) == GPIO_PIN_RESET) key2 |= K3_Pin;
        if (HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin) == GPIO_PIN_RESET) key2 |= K4_Pin;
        if (key == key2)     return key;  // 返回有效按键
        else                 return 0;
        }
        return 0;
}
/* USER CODE END 2 */
```

FreeRTOS应用示例——添加驱动头文件

在**gpio.h**中添加LED和按键驱动的函数声明，以及流水灯状态变量的外部声明：

```
/* USER CODE BEGIN Prototypes */
void SetLeds(uint8_t dat);
uint16_t ScanKey(void);
/* USER CODE END Prototypes */
```

准备添加流水灯控制的逻辑功能代码，打开MDK左侧工程文件列表中的**freertos.c**文件，在文件开头添加包含gpio.h头文件：

```
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "gpio.h"
/* USER CODE END Includes */
/* USER CODE BEGIN PM */
// 定义4个按键组合掩码
#define KEY_MASK  (K1_Pin | K2_Pin | K3_Pin | K4_Pin)
/* USER CODE END PM */
```

FreeRTOS应用示例——编写默认任务逻辑功能

在**freertos.c**中修改StartDefaultTask任务函数如下：

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  uint8_t sta = 0x01;       // 流水灯初始状态（L1亮）
  uint8_t dir = 1;          // 流水灯初始方向（1：右行，0：左行）
  for(;;) {
    SetLeds(sta);           // 流水灯亮灯
    // 等待按键消息
    int key = osThreadFlagsWait(KEY_MASK, osFlagsWaitAny, 0);
    if (key == K1_Pin)      // K1按下
      dir = !dir;           // 流水灯反向
    if (dir)    sta = (sta < 0x80) ? (sta << 1) : 0x01;// 右行流水灯变化
    else        sta = (sta > 0x01) ? (sta >> 1) : 0x80; // 左行流水灯变化
    osDelay(100);           // 流水灯变化时间间隔0.1秒
  }
  /* USER CODE END StartDefaultTask */
}
```

FreeRTOS应用示例——编写按键任务逻辑功能

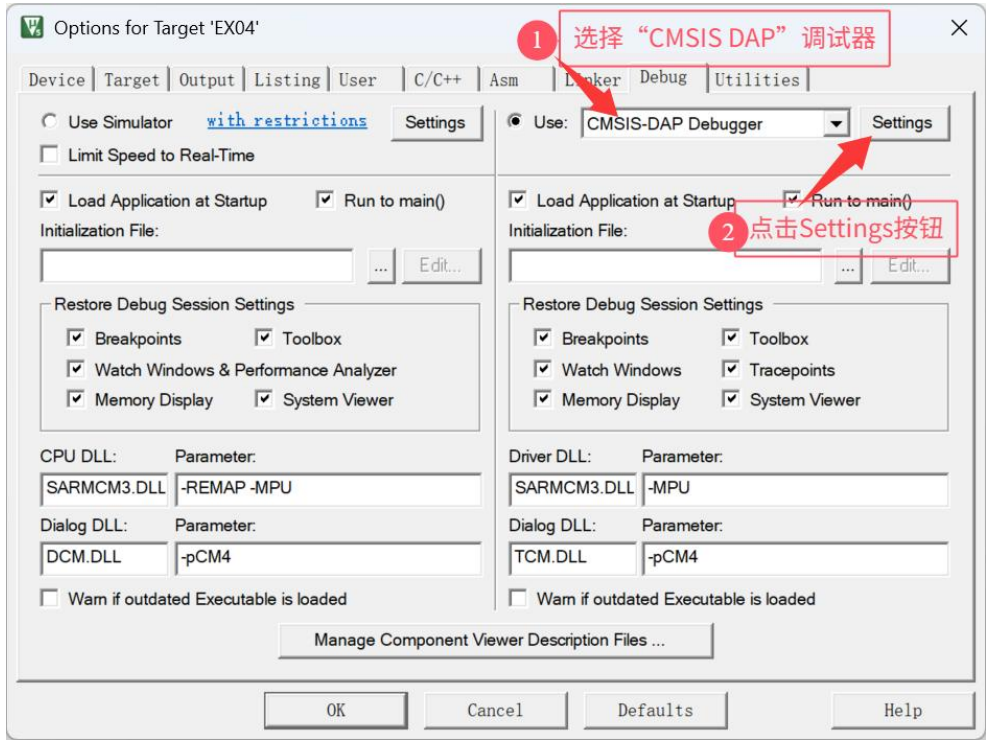
在**freertos.c**中修改StartKeyTask任务函数如下：

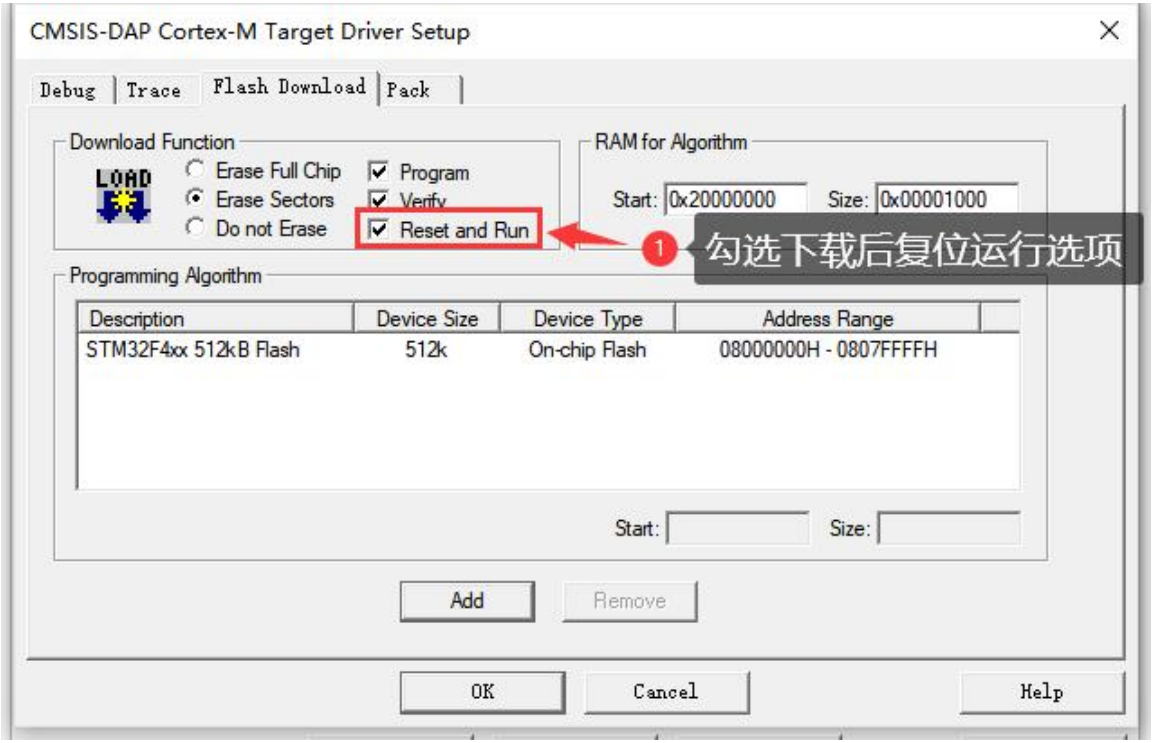
```
void StartKeyTask(void *argument) {
  /* USER CODE BEGIN StartKeyTask */
  uint8_t brun = 1;                    // 流水灯工作
  for(;;) {
    uint8_t key = ScanKey();           // 扫描按键键码
    if (key > 0) {                     // 有按键按下
      osThreadFlagsSet(defaultTaskHandle, key); // 向默认任务发送key通知
      while (ScanKey() > 0);           // 等待按键放开，防止按键连按
      if (K3_Pin == key) { brun = !brun;                    // 翻转流水灯工作状态
        if (brun) osThreadResume(defaultTaskHandle);        // 恢复运行流水灯
        else osThreadSuspend(defaultTaskHandle);            // 暂停流水灯工作
      }
    }
    osDelay(10);
  }
  /* USER CODE END StartKeyTask */
}
```

FreeRTOS应用示例——编译、下载、测试

添加代码完成后，连接学习板，参考下图进行MDK工程设置，确认调试器连接正常后，回到MDK主界面，按键盘F7快捷键编译工程。

编译成功后单击工具条上的“Download”按钮（快捷键F8）下载程序，就可以看到学习板上的流水灯效果，按下学习板上的K1、K3按键，观察按键功能是否正常。





实验内容

1. 示例工程实践练习

参照本次实验讲义内容，创建并复现EX04示例工程，最后在学习板上测试通过。

2. 任务应用练习

参考教材内容，修改FreeRTOS.c文件中的两个任务函数代码，为流水灯添加速度控制效果，用K1、K4按键实现流水灯速度加速和减速功能，用K2实现流水灯方向切换功能、K3实现流水灯暂停和恢复功能。

3. 到学习通上的对应课程作业中，完成提交本次实验的**课堂作业**。

思考问题：

在实验内容2基础上，修改K3按键功能，实现K3按住时流水灯暂停，K3松开时流水灯恢复。禁止定义全局变量，禁止使用任务暂停和恢复函数。

下一次实验05

RTOS任务间通信实验

请提前做好预习